PYTHON NOTES - 1. BASICS

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

1. Introduction

- Python is a high-level, interpreted, general-purpose programming language.
- Created by Guido van Rossum and released in 1991.
- Python is designed to be easy to read and write.
- It supports multiple programming paradigms : procedural, object-oriented and functional.



2. Features

- Easy to Learn
  Simple syntax, similar to English.
- Interpreted Language
  No need to compile, run line by line.
- High-level Language
  Focus on logic, not low-level details.
- Dynamically Typed
  No need to declare data type.
- Portable
  Write once, run anywhere.
- Extensive Libraries
  Large standard library for many tasks.
- Open Source
  Free to use and distribute.

3. Syntax

- Python syntax is clean and easy to read.
- It uses indentation (spaces or tabs) to define blocks of code.
- Case-sensitive language.

Example :

```
if True :
    print ("Hello, Python!")
else :
    print ("Goodbye!")
```

Note :
Indentation is very important in Python. It defines the scope of code blocks.

4. Comments

- Comments are used to explain code.
- They are ignored by the interpreter.

Types of Comments :

A. Single-line Comment

```
# This is a single-line comment
```

B. Multi-line Comment

```
'''
This is a
multi-line comment.
It can span
multiple lines.
'''
```

(Also, we can use multiple # for multi-line comments.)

5. Variables

- Variables are used to store data.
- No need to declare type explicitly.
- Created when value is assigned.

Example :

```
name    =  "Abhi"
age     =  21
pi      =  3.14
is_student  =  True
```

Python is dynamically typed.
The type is determined at runtime.

6. Keywords

- Keywords are reserved words in Python.
- They have special meaning and cannot be used as identifiers.
- Example keywords :

False	None	True	and	as
assert	break	class	continue	def
del	elif	else	except	finally
for	from	global	if	import
in	is	lambda	nonlocal	not
or	pass	raise	return	try
while	with	yield		

7. Identifiers

- Identifiers are names given to variables, functions, classes, etc.
- Rules for Identifiers :
  1. Can contain letters (a-z, A-Z), digits (0-9) and underscore (_).
  2. Must start with a letter or underscore.
  3. Cannot start with a digit.
  4. Cannot be a keyword.
  5. Case-sensitive (age, Age and AGE are different).

Valid Identifiers

```
my_var
_value
count1
total_sum
userName
```

Invalid Identifiers

```
1count
my-var
total sum
for
class
```

★ Strong basics build strong programs. Keep Practicing !

Think Smart. Code Better. Optimize Always !



PYTHON NOTES - 2. DATA TYPES

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

1. Numbers

- Python has three numeric types : int, float and complex.
- int → whole numbers
- float → decimal numbers
- complex → numbers with real and imaginary part.

Examples :

```
a = 10          # int
b = 3.14        # float
c = 2 + 3j      # complex
```

Numbers are the building blocks for all calculations in Python.

2. Strings

- String is a sequence of characters enclosed in single, double or triple quotes.
- Strings are immutable (cannot be changed).

Examples :

```
s1 = "Hello"
s2 = 'Python'
s3 = """Multi
     line
     string"""
```

Common Operations :

```
s = "Hello"
len(s)              # 5
s[0]                # 'H'
s + " World"        # "Hello World"
s.upper()           # "HELLO"
s.lower()           # 'hello'
s.replace('l','x')  # "Hexxo"
```

3. Boolean

- Boolean type has two values : True or False.
- Used in conditions and logical operations.

Examples :

```
a = True
b = False
print(a, b)      # True False
```

4. None

- None represents the absence of value.
- It is a special data type of its own.
- Often used to initialize variables.

Example :

```
x = None
print(x)      # None
```

★ None is not 0, False or empty, It means "nothing".

5. Type Casting

- Type casting is converting one data type to another.
- This helps when we need to perform operations between different types.

A. Implicit Type Casting

- Python automatically converts smaller type to larger type.
- Example :

```
a = 5          # int
b = 2.5        # float
c = a + b      # result → 7.5 (float)
```

B. Explicit Type Casting

- We can explicitly convert one type to another using built-in functions.
- Common functions : int(), float(), str(), bool()

Function	Description	Example	Output
int()	Converts value to integer	int(10.7) int("20")	10 20
float()	Converts value to float	float(5) float("3.14")	5.0 3.14
str()	Converts value to string	str(100) str(3.14)	'100' '3.14'
bool()	Converts value to boolean	bool(1) bool(0) bool("")	True False False

✡ Truthy and Falsy Values

- In Python, some values are considered True (Truthy) and others False (Falsy).

Truthy Values

```
True, 1, -1, "hello", [1,2],
{1: 'a'}, (1,), 3.14, etc.
```

Falsy Values

```
False, 0, None, "", [], (), {}
(empty collections), etc.
```

Examples (All Data Types Together)

Value	Type	Example
10	int	x = 10
-25	int	y = -25
3.1415	float	pi = 3.1415
-0.5	float	temp = -0.5
2 + 5j	complex	c = 2 + 5j
'Python'	str	name = 'Python'
"Hello World"	str	msg = "Hello World"
True	bool	is_active = True
False	bool	is_done = False
None	NoneType	value = None

Quick Type Check

Use type() to check data type.

```
x = 10
type(x)          # <class 'int'>
y = 3.14
type(y)          # <class 'float'>
s = "Hi"
type(s)          # <class 'str'>
b = True
type(b)          # <class 'bool'>
n = None
type(n)          # <class 'NoneType'>
```

★ Remember :
Choose the right data type to save memory and avoid unexpected results.



Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 3. INPUT & OPERATORS

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

1. Input / Output

- Input is taken using `input()` function.

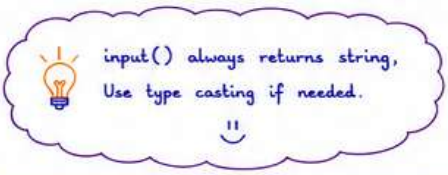
Example :

```
name = input("Enter your name: ")
age = int(input("Enter age: "))
print("Hello", name)
print("Age is", age)
```

- Output is shown using `print()` function.
- `print()` can print multiple values.

Example :

```
a = 10
b = 20
print("Sum =", a + b)
print(a, b, sep=" | ")    # 10 | 20
```



2. Arithmetic Operators

- Used to perform mathematical operations.

Operator	Name	Example	Result
+	Addition	5 + 3	8
-	Subtraction	5 - 3	2
*	Multiplication	5 * 3	15
/	Division	5 / 2	2.5
//	Floor Division	5 // 2	2
%	Modulus	5 % 2	1
**	Exponent	5 ** 3	125
-	Unary Minus	-5	-5
+	Unary Plus	+5	5

Example :

```
a = 10
b = 3
print(a + b, a - b, a * b)    # 13 7 30
print(a / b, a // b, a % b)   # 3.333.. 3 1
print(a ** b)                 # 1000
```

3. Relational (Comparison) Operators

- Used to compare two values.

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	5 > 3	True
<	Less than	5 < 3	False
>=	Greater than or equal to	5 >= 5	True
<=	Less than or equal to	5 <= 4	False

Example :

```
a = 5
b = 3
print(a > b)     # True
print(a == b)    # False
print(a != b)    # True
```

★ Relational operators always return True or False.

4. Logical Operators

- Used to combine conditional statements.

Operator	Meaning	Example	Result
and	True if both	True and False	False
or	True if any one	True or False	True
not	Reverse the result	not True	False

Example :

```
a = True
b = False
print(a and b)     # False
print(a or b)      # True
print(not a)       # False
```

5. Assignment Operators

- Used to assign values to variables.

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
//=	x //= 3	x = x // 3
%=	x %= 3	x = x % 3
**=	x **= 3	x = x ** 3

Example :

```
x = 10
x += 5     # x = 15
x *= 2     # x = 30
```

6. Bitwise Operators

- Works on bits and performs bit-level operations.

Operator	Meaning	Example	Result
&	AND	5 & 3	1
|	OR	5 | 3	7
^	XOR	5 ^ 3	6
~	NOT	~5	-6
<<	Left Shift	5 << 1	10
>>	Right Shift	5 >> 1	2

Example :

```
a = 5          # 0101
b = 3          # 0011
print(a & b)   # 1    (0001)
print(a | b)   # 7    (0111)
print(a ^ b)   # 6    (0110)
print(~a)      # -6
print(a << 1)  # 10   (1010)
print(a >> 1)  # 2    (0010)
```

7. Membership Operators

- Used to test sequence membership.

Operator	Meaning	Example	Result
in	True if value is present	'a' in 'cat'	True
not in	True if value is not present	'z' not in 'cat'	True

Example :

```
s = "hello"
print('h' in s)      # True
print('x' in s)      # False
print('lo' in s)     # True
print('lo' not in s) # False
```

★ Works with strings, lists, tuples, sets, dictionaries.

8. Identity Operators

- Used to compare memory location (identity).

Operator	Meaning	Example	Result
is	True if both variables are the same object	a is b	True/False
is not	True if both variables are not the same object	a is not b	True/False

Example :

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
print(a is b)      # True    (same object)
print(a is c)      # False   (different object)
print(a is not c)  # True
```

is checks identity (address), == checks equality (value). Remember the difference !

Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 4. CONDITIONAL STATEMENTS & LOOPS

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

★ PART A : CONDITIONAL STATEMENTS

1. if Statement

- Executes a block if the condition is True.
- If condition is False, block is skipped.

Syntax :

```
if condition:
    # block of code
```

Example :

```
x = 10
if x > 5:
    print("x is greater than 5")
```

2. if - else Statement

- Executes one block if condition is True otherwise executes the else block.

Syntax :

```
if condition:
    # block if True
else:
    # block if False
```

Example :

```
x = 3
if x % 2 == 0:
    print("Even")
else:
    print("Odd")
```

3. elif Statement

- Used with if-else to check multiple conditions.
- elif stands for else if.

Syntax :

```
if condition1:
    # block
elif condition2:
    # block
else:
    # block
```

Example :

```
marks = 85
if marks >= 90:
    print("A Grade")
elif marks >= 75:
    print("B Grade")
elif marks >= 50:
    print("C Grade")
else:
    print("Fail")
```

4. Nested if Statement

- An if...else statement inside another if...else.
- Used to handle complex conditions.

Syntax :

```
if condition1:
    if condition2:
        # block
    else:
        # block
else:
    # block
```

Example :

```
x = 10
if x > 0:
    if x % 2 == 0:
        print("Positive Even")
    else:
        print("Positive Odd")
else:
    print("Negative or Zero")
```

Conditional statements help the program make decisions.
Choose the right path

★ PART B : LOOPS

1. for Loop

- Used to iterate over a sequence (list, tuple, string, range).

Syntax :

```
for variable in sequence:
    # block of code
```

Example :

```
for i in range(1, 6):
    print(i)
# Output : 1 2 3 4 5
```

2. while Loop

- Repeats a block as long as the condition is True.

Syntax :

```
while condition:
    # block of code
```

Example :

```
i = 1
while i <= 5:
    print(i)
    i += 1
# Output : 1 2 3 4 5
```

3. Nested Loops

- A loop inside another loop.
- Useful for patterns and matrices.

Example :

```
for i in range(1, 4):
    for j in range(1, 4):
        print(i * j, end=" ")
    print()
```

```
# Output :
1 2 3
2 4 6
3 6 9
```

4. break Statement

- Terminates the loop immediately.

Example :

```
for i in range(1, 6):
    if i == 3:
        break
    print(i)
# Output : 1 2
```

5. continue Statement

- Skips the current iteration and continues with next.

Example :

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
# Output : 1 2 4 5
```

6. pass Statement

- Does nothing. Used as a placeholder for future code.
- Helps avoid syntax errors.

Example :

```
for i in range(3):
    pass  # To be implemented later
```

★ Loops reduce repetition and save time & effort!

Think Smart. Code Better. Optimize Always!

PYTHON NOTES - 5. STRINGS & LISTS

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

★ PART A : STRINGS

1. Indexing

- Each character in a string has an index.
- Index starts from 0.

Example :

```
s = "Python"
Index :  0  1  2  3  4  5
Char  :  P  y  t  h  O  n

s[0] = 'P'        s[-1] = 'n'
s[1] = 'y'        s[-2] = 'o'
```

Strings are immutable (cannot be changed).

2. Slicing

- Used to extract a part of string.
- Syntax : `s[start : end : step]`

Example :

```
s = "Hello, World!"
s[0:5]    = "Hello"
s[7:12]   = "World"
s[:]      = "Hello, World!"
s[::2]    = "Hlo ol!"
s[::-1]   = "!dlroW ,olleH"
```

Notes :

- end index is excluded.
- default start = 0
  default end = len(s)
  default step = 1

3. String Methods

- Strings have many built-in methods.

Example :

Method	Description	Example	Output
s.upper()	Converts to uppercase	'py'.upper()	'PY'
s.lower()	Converts to lowercase	'PY'.lower()	'py'
s.strip()	Removes spaces	' hi '.strip()	'hi'
s.replace(old, new)	Replaces substring	'a-b'.replace('-','_')	'a_b'
s.split(sep)	Split into list	'a,b,c'.split(',')	['a','b','c']
s.find(x)	Returns index of x	'hello'.find('e')	1
s.count(x)	Counts occurrences	'banana'.count('a')	3

4. String Formatting

- Used to insert values into strings.

Using % operator

```
name = "Abhi"
age = 20
s = "My name is %s
and age is %d" %
(name, age)
# My name is Abhi
# and age is 20
```

Using format()

```
name = "Abhi"
age = 20
s = "My name is {}
and age is {}".format
(name, age)
# My name is Abhi
# and age is 20
```

Using f-string (Python 3.6+)

```
name = "Abhi"
age = 20
s = f"My name is {name}
and age is {age}"

# My name is Abhi
# and age is 20
```

5. f-Strings

- f-strings allow embedding expressions inside {}.
- Fast, readable and easy.

Example :

```
name = "Abhi"
age = 20
marks = 95.5
s = f"{name} is {age} years old and scored {marks}%"
# Abhi is 20 years old and scored 95.5%
```

★ f-strings are the modern and best way to format strings !

★ PART B : LISTS

1. Creation

- Lists are ordered, mutable collections.

Example :

```
empty_list = []
nums = [1, 2, 3, 4, 5]
mixed = [1, 'hi', 3.5, True]
nested = [1, [2, 3], 'abc']
```

Lists can store different data types together.

2. Indexing & Slicing

- Access elements using index and slicing.

Indexing Example :

```
nums = [10, 20, 30, 40, 50]
Index :  0   1   2   3   4
nums[0]  = 10
nums[2]  = 30
nums[-1] = 50
nums[-2] = 40
```

Slicing Example :

```
nums[1:4]   = [20, 30, 40]
nums[:3]    = [10, 20, 30]
nums[2:]    = [30, 40, 50]
nums[::2]   = [10, 30, 50]
nums[::-1]  = [50, 40, 30, 20, 10]
```

3. List Methods

- Common methods to work with lists.

Method	Description	Example	Output
append(x)	Adds element at end	lst.append(5)	[1,2,3,5]
insert(i, x)	Inserts at index i	lst.insert(1, 9)	[1,9,2,3]
extend(iterable)	Adds all elements	lst.extend([4,5])	[1,2,3,4,5]
remove(x)	Removes first occ.	lst.remove(2)	[1,3]
pop(i)	Removes & returns	lst.pop()	3
index(x)	Returns index of x	lst.index(2)	1
count(x)	Counts occurrences	lst.count(1)	2
sort()	Sorts the list	lst.sort()	[1,2,3]
reverse()	Reverses the list	lst.reverse()	[3,2,1]
clear()	Removes all items	lst.clear()	[]

4. Nested Lists

- Lists can contain other lists.

Example :

```
matrix = [[1, 2, 3],[4, 5, 6],[7, 8, 9]]
matrix[0]      = [1, 2, 3]
matrix[1][2]   = 6
matrix[2][0]   = 7
matrix[1][:2]  = [4, 5]
```

Matrix View :

1	2	3
4	5	6
7	8	9

Row = outer index
Column = inner index

5. Loop Control in Loops

break Statement

- Terminates the loop completely.

Example :

```
for i in range(5):
    if i == 3:
        break
    print(i)
# Output: 0 1 2
```

continue Statement

- Skips current iteration and continues.

Example :

```
for i in range(5):
    if i == 2:
        continue
    print(i)
# Output: 0 1 3 4
```

pass Statement

- Does nothing. Used as placeholder.

Example :

```
for i in range(3):
    if i == 1:
        pass
    print(i)
# Output: 0 1 2
```

Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 6. TUPLES

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

1. Creation

- Tuple is an ordered, immutable collection.
- Tuples are faster than lists.
- Elements are written inside parentheses ().

Example :

```
empty_tuple = ()
numbers = (1, 2, 3, 4)
mixed = (1, "hi", 3.14, True)
single = (5,)          # note the comma
nested = (1, (2, 3), 'abc')
```

Tuples are immutable. Once created, cannot be changed.

2. Indexing

- Access elements using index.
- Index starts from 0.

Example :

```
t = ('p', 'y', 't', 'h', 'o', 'n')
Index : 0  1  2  3  4  5
t[0]  = 'p'
t[3]  = 'h'
t[-1] = 'n'
t[-3] = 'h'
```

★ Negative index starts from the end.

3. Slicing

- Used to extract a part of tuple.
- Syntax : `t[start : end : step]`

Example :

```
t = (10, 20, 30, 40, 50, 60)
t[1:4]   = (20, 30, 40)
t[:3]    = (10, 20, 30)
t[3:]    = (40, 50, 60)
t[::2]   = (10, 30, 50)
t[::-1]  = (60, 50, 40, 30, 20, 10)
```

Notes :

- end index is excluded.
- default start = 0
- default end = len(t)
- default step = 1

4. Tuple Methods

- Tuples have only two built-in methods.

Method	Description	Example	Output
count(x)	Returns number of occurrences of x	t = (1, 2, 2, 3, 2) t.count(2)	3
index(x)	Returns index of first occurrence of x	t = (10, 20, 30, 20) t.index(20)	1

Example :

```
t = (4, 5, 6, 5, 7, 5)
print(t.count(5))      # 3
print(t.index(6))      # 2
```

★ No append(), remove(), pop() in tuples!

5. Packing & Unpacking

- Packing ⟶ Combining values into a tuple.
- Unpacking ⟶ Extracting values from a tuple.

A. Packing

- Values are automatically packed into a tuple.

Example :

```
t = 1, 2, 3, 4, 5      # tuple packing
print(t)               # (1, 2, 3, 4, 5)
```

OR

```
t = (10, 20, 30)
print(type(t))         # <class 'tuple'>
```

Packing is creating a tuple, Unpacking is extracting values from it.

★ Tuples are used when data should not be changed. They are safe and efficient!

B. Unpacking

- Values are assigned to multiple variables.

Example :

```
a, b, c = (10, 20, 30)
print(a, b, c)         # 10 20 30
```

Extended Unpacking (Python 3+)

Example :

```
t = (1, 2, 3, 4, 5, 6)
a, b, *rest, last = t
print(a)               # 1
print(b)               # 2
print(rest)            # [3, 4, 5]
print(last)            # 6
```

Rules :

- Number of variables must match number of values.
- Use * to collect remaining values in a list.
- Only one * is allowed in unpacking.

Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 7. SETS & DICTIONARIES

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

★ PART A : SETS

1. Creation

- Sets are unordered collections of unique elements.
- Duplicates are not allowed.
- Elements must be immutable (no list, dict or set).

Sets are unordered and do not allow duplicate values.

Example :

```
empty_set = set()                    # empty set
numbers = {1, 2, 3, 4, 5}
mixed = {1, 'hi', 3.14, True}
letters = set('python')              # {'p', 'y', 't', 'h', 'o', 'n'}
```

2. Set Methods

Method	Description	Example	Output
add(x)	Adds an element	s = {1, 2}; s.add(3)	{1, 2, 3}
update(iterable)	Adds multiple elements	s.update([3,4])	{1, 2, 3, 4}
remove(x)	Removes x, error if not found	s.remove(2)	{1, 3}
discard(x)	Removes x, no error if not found	s.discard(5)	{1, 2, 3}
pop()	Removes & returns an random element	s.pop()	2 (any one)
clear()	Removes all elements	s.clear()	set()
len(s)	Returns number of elements	len(s)	3

Example :

```
s = {10, 20, 30}
s.add(40)
s.update([50, 60])
s.remove(20)
print(s)        # {10, 30, 40, 50, 60}
```

★ pop() removes any random element from the set. It is not fixed.

3. Union

- Returns a set containing all elements from both sets.

Example :

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a | b)          # {1, 2, 3, 4, 5}
print(a.union(b))     # {1, 2, 3, 4, 5}
```

4. Intersection

- Returns a set containing common elements of both sets.

Example :

```
a = {1, 2, 3, 4}
b = {3, 4, 5}
print(a & b)                  # {3, 4}
print(a.intersection(b))      # {3, 4}
```

5. Difference

- Returns elements in first set that are not in second set.

Example :

```
a = {1, 2, 3, 4}
b = {3, 4, 5}
print(a - b)                  # {1, 2}
print(a.difference(b))        # {1, 2}
print(b - a)                  # {5}
```

Use sets when you need unique items and fast membership checks. They are powerful for mathematical operations !

★ PART B : DICTIONARIES

1. Keys & Values

- Dictionaries store data in key : value pairs.
- Keys must be unique and immutable.
- Values can be of any data type.

Think of dictionary like real dictionary (key to find value).

Example :

```
empty_dict = {}
student = {'name': 'Abhi', 'age': 20, 'grade': 'A'}
marks = dict(math=95, eng=88, sci=90)
print(student['name'])        # Abhi
print(student['age'])         # 20
```

2. Dictionary Methods

Method	Description	Example	Output
keys()	Returns all keys	d.keys()	dict_keys(['a', 'b'])
values()	Returns all values	d.values()	dict_values([1, 2])
items()	Returns key-value pairs	d.items()	dict_items([('a',1),('b',2)])
get(key, default)	Returns value for key	d.get('a') d.get('x', 0)	1 0
update(other_dict)	Updates with key-value	d.update({'b':3})	{'a': 1, 'b': 3}
pop(key, default)	Removes key & returns value	d.pop('a')	1
popitem()	Removes last inserted key-value pair	d.popitem() (Python 3.7+)	('b', 2)
clear()	Removes all items	d.clear()	{}
len(d)	Returns number of items	len(d)	2

Example :

```
d = {'a': 1, 'b': 2}
d['c'] = 3
d.update({'b': 20})
print(d)              # {'a': 1, 'b': 20, 'c': 3}
print(d.keys())       # dict_keys(['a', 'b', 'c'])
print(d.values())     # dict_values([1, 20, 3])
```

★ Use get() to avoid KeyError. It returns default value if key not found.

3. Nested Dictionaries

- A dictionary can contain another dictionary as a value.

Example :

```
student = {
  'name': 'Abhi',
  'age': 20,
  'marks': {'math': 95, 'eng': 88, 'sci': 90},
  'address': {'city': 'Delhi', 'pin': 110001}
}
print(student['name'])                # Abhi
print(student['marks']['math'])       # 95
print(student['address']['city'])     # Delhi
```

Visual View :

name	age	marks	address
'Abhi'	20	{ }	{ }

math	eng	sci
95	88	90

city	pin
'Delhi'	110001

Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 8. DICTIONARY & COMPREHENSIONS

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

★ PART A : DICTIONARY (DEEP DIVE)

1. What is Dictionary ?

- Dictionary stores data in key : value pairs.
- Keys must be unique and immutable.
- Values can be any data type.

Dictionaries are very fast for lookups using keys.

2. Creation

Example :

```
empty_dict = {}                    # empty dictionary
student = {'name': 'Abhi','age': 20, 'grade': 'A'}
mixed = dict(id=101, active=True, score=95.5)
from_pairs = dict([('a', 1), ('b', 2)])
```

Notes :

- Keys must be unique.
- Order is preserved (Python 3.7+).

3. Accessing Elements

Example :

```
d = {'name': 'Abhi', 'age': 20, 'city': 'Delhi'}
d['name']              # Abhi
d.get('age')           # 20
d.get('pin', 'N/A')    # N/A  (default)
'city' in d            # True
```

★ Use get() to avoid KeyError if key is not present.

4. Updating & Adding Elements

Example :

```
d = {'name': 'Abhi'}
d['age'] = 20                  # Add new key-value
d['name'] = 'Abhishek'         # Update existing key
d.update({'city': 'Delhi', 'pin': 110001})
d.setdefault('country', 'India')  # Adds only if not exists
```

5. Deleting Elements

Example :

```
d = {'a': 1, 'b': 2, 'c': 3}
del d['b']             # Remove by key
val = d.pop('c')       # Remove & return value (3)
d.popitem()            # Remove last inserted item
d.clear()              # Remove all items -> {}
```

Notes :

- del removes by key.
- pop() removes & returns value.
- popitem() removes last inserted pair.

6. Looping in Dictionary

Example :

```
d = {'a': 1, 'b': 2, 'c': 3}
# Loop over keys
for k in d:
    print(k, end=' ')          # a b c
# Loop over values
for v in d.values():
    print(v, end=' ')          # 1 2 3
# Loop over key-value pairs
for k, v in d.items():         # a 1
    print(k, v)                # b 2
                               # c 3
```

items() returns (key, value) pairs. Very useful in loops.

★ PART B : COMPREHENSIONS

1. What are Comprehensions ?

- Comprehensions provide a concise way to create sequences (lists, sets, dicts) in a single line.
- They are more readable and usually faster.

Comprehensions make code short, clean and pythonic !

2. List Comprehension

Syntax : [expression for item in iterable]

Example :

```
# Squares of numbers
squares = [x*x for x in range(1, 6)]          # [1, 4, 9, 16, 25]
# With condition
evens = [x for x in range(1, 11) if x % 2 == 0]   # [2, 4, 6, 8, 10]
# Using if-else
result = ['Even' if x%2==0 else 'Odd' for x in range(1,6)]
                          # ['Odd', 'Even', 'Odd', 'Even', 'Odd']
```

3. Set Comprehension

Syntax : { expression for item in iterable }

Example :

```
squares = {x*x for x in range(1, 6)}              # {1, 4, 9, 16, 25}
unique_chars = {ch for ch in 'abracadabra'}       # {'a', 'b', 'r', 'c', 'd'}
vowels = {ch for ch in 'education' if ch in 'aeiou'}   # {'e', 'u', 'a', 'i', 'o'}
```

4. Dictionary Comprehension

Syntax : { key_expr : value_expr for item in iterable }

Example :

```
# Number : Square
num_sq = {x: x*x for x in range(1, 6)}
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
# Character : ASCII value
char_code = {ch: ord(ch) for ch in 'ABC'}   # {'A': 65, 'B': 66, 'C': 67}
# With condition
even_sq = {x: x*x for x in range(1, 11) if x%2 == 0}
                          # {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

5. Conditional Comprehension

Syntax : [expression for item in iterable if condition]

Example :

```
# List of even numbers
evens = [x for x in range(1, 21) if x % 2 == 0]
# Numbers greater than 10
gt_10 = [x for x in range(1, 21) if x > 10]
# Dictionary with condition
sq_gt_10 = {x: x*x for x in range(1, 21) if x > 10}
          # {11: 121, 12: 144, ..., 20: 400}
```

Notes :

- Comprehensions improve readability.
- Often faster than loops.
- Best used for simple to moderate logic.

Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 9. COMPREHENSION & FUNCTIONS

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

★ PART A : COMPREHENSIONS

1. What is Comprehensions ?

- Comprehensions provide a concise way to create sequences (lists, sets, dicts) in a single line.
- They are more readable and usually faster.

Write less code, do more. That's the power of comprehensions

2. List Comprehension

Syntax : [expression for item in iterable]

Example :

```
# Squares
squares = [x*x for x in range(1, 6)]                    # [1, 4, 9, 16, 25]
# Even numbers
evens = [x for x in range(1, 11) if x % 2 == 0]         # [2, 4, 6, 8, 10]
# Using if-else
result = ['Even' if x%2==0 else 'Odd' for x in range(1,6)]
                                    # ['Odd', 'Even', 'Odd', 'Even', 'Odd']
```

3. Set Comprehension

Syntax : { expression for item in iterable }

Example :

```
unique_chars = {ch for ch in 'abracadabra'}             # {'a', 'b', 'r', 'c', 'd'}
vowels = {ch for ch in 'education' if ch in 'aeiou'}    # {'e', 'u', 'a', 'i', 'o'}
squares = {x*x for x in range(1, 6)}                    # {1, 4, 9, 16, 25}
```

4. Dictionary Comprehension

Syntax : { key_expr : value_expr for item in iterable }

Example :

```
# Number : Square
num_sq = {x : x*x for x in range(1, 6)}     # {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
# Character : ASCII
char_code = {ch : ord(ch) for ch in 'ABC'}  # {'A': 65, 'B': 66, 'C': 67}
# With condition
even_sq = {x : x*x for x in range(1, 11) if x % 2 == 0}
                    # {2: 4, 4: 16, 6: 36, 8: 64, 10: 100}
```

5. Conditional Comprehension

Syntax : [expression for item in iterable if condition]

Example :

```
# List of even numbers
evens = [x for x in range(1, 21) if x % 2 == 0]
# [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
# Numbers greater than 10
gt_10 = [x for x in range(1, 21) if x > 10]
# [11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
# Dictionary with condition
sq_gt_10 = {x : x*x for x in range(1, 21) if x > 10}
        # {11: 121, 12: 144, ..., 20: 400}
```

Notes :

- Comprehensions improve readability.
- Often faster than loops.
- Best used for simple to moderate logic.

★ PART B : FUNCTIONS

1. What is Function ?

- A function is a block of reusable code.
- Functions help in code reusability and modularity.

Functions make your code reusable, organized and powerful !

2. Parameters vs Arguments

- Parameters are variables listed inside the function definition.
- Arguments are the actual values passed while calling the function.

Parameter (in function)

```
def greet(name):
    print(f"Hello, {name}!")
# name is a parameter
```

Argument (in function call)

```
greet('Abhi')
# 'Abhi' is an argument
```

3. Return Statement

- return sends a value back from the function.
- If no return, function returns None by default.

Example :

```
def add(a, b):
    return a + b
res = add(5, 3)
print(res)          # 8
```

Use return when you want a result from the function.

4. Default Arguments

- Provide default values to parameters.
- If argument not passed, default is used.

Example :

```
def greet(name, msg="Welcome!"):
    print(f"Hello {name}, {msg}")
greet('Abhi')                   # Hello Abhi, Welcome!
greet('Abhi', 'Good Morning!')  # Hello Abhi, Good Morning!
```

Defaults make functions flexible and easier to use.

5. *args (Arbitrary Positional Arguments)

- Allows function to accept any number of positional arguments.

Example :

```
def add_all(*nums):
    total = 0
    for n in nums:
        total += n
    return total

print(add_all(1, 2, 3))         # 6
print(add_all(1, 2, 3, 4, 5))   # 15
print(add_all(10))              # 10
```

Here, *nums becomes a tuple.

6. **kwargs (Arbitrary Keyword Arguments)

- Allows function to accept any number of keyword arguments.

Example :

```
def info(**details):
    for k, v in details.items():
        print(f"{k} : {v}")

info(name='Abhi', age=20, city='Delhi')
# name : Abhi
# age : 20
# city : Delhi
```

Here, **details becomes a dictionary.

★ Quick Summary

- Parameters are in function definition.
- Arguments are in function call.
- return sends value back.
- *args for many positional args.
- **kwargs for many keyword args.

Well used functions can make your code clean & powerful !

Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 10. ADVANCED FUNCTIONS & RECURSION

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

★ PART A : ADVANCED FUNCTIONS

1. Lambda (Anonymous Function)

- Lambda creates small anonymous functions.
- Useful for short, one-time use functions.
- Syntax : lambda arguments : expression

Lambda functions are small but powerful !

Example :

```
# Add two numbers
add = lambda a, b: a + b
print(add(5, 3))          # 8

# Square of a number
square = lambda x: x * x
print(square(4))          # 16

# Lambda with multiple arguments
mul = lambda x, y, z: x * y * z
print(mul(2, 3, 4))       # 24
```

2. map()

- map() applies a function to all items in an iterable.
- Returns a map object (convertible to list).

Syntax : map(function, iterable)

Example :

```
nums = [1, 2, 3, 4, 5]

# Square each number
squares = list(map(lambda x: x*x, nums))
print(squares)              # [1, 4, 9, 16, 25]

# Convert to upper case
words = ['py', 'pthon', 'rocks']
upper = list(map(str.upper, words))
print(upper)          # ['PY', 'THON', 'ROCKS']
```

★ map() makes your code shorter and cleaner !

3. filter()

- filter() filters elements based on a condition.
- Returns only elements for which function returns True.

Syntax : filter(function, iterable)

Example :

```
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Even numbers
is_even = list(filter(lambda x: x % 2 == 0, nums))
print(is_even)              # [2, 4, 6, 8, 10]

# Words longer than 3 characters
words = ['hi', 'python', 'is', 'awesome', 'code']
long_words = list(filter(lambda w: len(w) > 3, words))
print(long_words)       # ['python', 'awesome', 'code']
```

filter() keeps only what matches the condition !

4. reduce()

- reduce() applies a function cumulatively to the items of an iterable.
- Returns a single accumulated result.
- reduce() is in functools module.

Syntax : reduce(function, iterable[, initializer])

```
from functools import reduce
nums = [1, 2, 3, 4]

# Sum of all numbers
sum_all = reduce(lambda x, y: x + y, nums)
print(sum_all)              # 10

# Product of all numbers
prod_all = reduce(lambda x, y: x * y, nums, 1)
print(prod_all)             # 24
```

★ reduce() reduces the entire iterable to a single value !

★ PART B : RECURSION

1. What is Recursion ?

- A function calling itself to solve a smaller instance of the same problem.
- Recursion breaks a problem into smaller subproblems.

Recursion is when a function trusts itself to solve smaller problems !

2. Base Case & Recursive Case

Term	Description
Base Case	• The condition when recursion stops. • Prevents infinite recursion. • Returns the final result for the simplest case.
Recursive Case	• The condition when function calls itself with a smaller input. • Moves problem towards base case.

3. Factorial (Using Recursion)

Problem : n! = n × (n-1) × (n-2) × ... × 1
0! = 1

Example :

```
def factorial(n):
    # Base Case
    if n == 0 or n == 1:
        return 1
    # Recursive Case
    return n * factorial(n - 1)

# Test
print(factorial(5))         # 120
print(factorial(0))         # 1
```

Dry Run for factorial(4)

```
factorial(4)
= 4 * factorial(3)
= 4 * (3 * factorial(2))
= 4 * (3 * (2 * factorial(1)))
= 4 * (3 * (2 * 1))
= 4 * (3 * 2)
= 4 * 6
= 24
```

4. Fibonacci Series (Using Recursion)

Series : 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

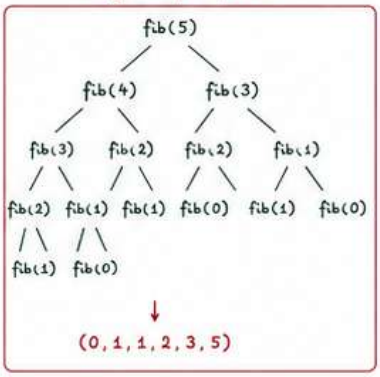
Rule : fib(n) = fib(n-1) + fib(n-2)
fib(0) = 0, fib(1) = 1

Example :

```
def fib(n):
    # Base Case
    if n == 0:
        return 0
    if n == 1:
        return 1
    # Recursive Case
    return fib(n - 1)+ fib(n - 2)

# Print first 10 Fibonacci numbers
for i in range(10):
    print(fib(i), end=' ')
# Output: 0 1 1 2 3 5 8 13 21 34
```

Dry Run for fib(5)



Important Notes

- Always define a base case, otherwise recursion will never stop (infinite recursion).
- Recursive solutions are elegant but may use more memory (stack space).
- Some problems are naturally recursive (trees, divide & conquer, DP problems).

★ Break it down, Solve small, Build big !

Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 11. MODULES & PACKAGES

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

★ What are Modules & Packages ?

- A module is a Python file containing Python definitions and statements (.py file).
- A package is a collection of modules in a directory (with __init__.py file).
- They help in code reusability, organization and making large programs manageable.

Modules make your code modular. Packages make it structured !

Benefits

- Code Reusability → Use existing code.
- Better Organization → Split large code into modules.
- Easy Maintenance → Modify one module without affecting others.
- Namespace Management → Avoid name conflicts.

Don't reinvent the wheel. Import it !

1. import Statement

- Imports the entire module.
- Access members using module_name.member_name

Example : (using math module)

```
1  import math
2
3  print(math.sqrt(16))     # 4.0
4  print(math.pi)           # 3.141592653589793
5  print(math.ceil(2.3))    # 3
6  print(math.floor(2.8))   # 2
```

★ Use when you need many functions from the module.

Note :
- If module name is long, can use alias → import math as m
  Example : m.sqrt(25)

2. from ... import Statement

- Imports specific members from a module.
- Syntax : from module_name import name1, name2, ...

Example :

```
1  from math import sqrt, pi
2
3  print(sqrt(25))          # 5.0
4  print(pi)                # 3.141592653589793
```

★ Use when you need only some functions.

Note :
- Can import all using * (not recommended in large projects).
- Example : from math import *
- Then directly : sqrt(16), ceil(2.3) etc.

3. Built-in Modules

- Python comes with many built-in modules.
- No installation required.
- Some commonly used built-in modules :

Module	Description	Example Usage
math	Mathematical functions	import math
random	Generate random numbers	import random
datetime	Date and time handling	import datetime
os	Operating system interaction	import os
sys	System-specific parameters	import sys
json	Work with JSON data	import json

Use built-in modules for common tasks.

Example : (random module)

```
1  import random
2  print(random.randint(1, 10))          # Random integer between 1 and 10
3  print(random.choice(['a', 'b', 'c'])) # Random choice from list
```

4. Custom Modules

- We can create our own modules (.py files) and import them.
- Steps :

1. Create a Python file (e.g., mymodule.py)
2. Write functions / variables in it.
3. Import and use in another file.

mymodule.py

```
1  def greet(name):
2      return f"Hello, {name}!"
3
4  def add(a, b):
5      return a + b
```

main.py

```
1  import mymodule
2
3  print(mymodule.greet("Abhi"))    # Hello, Abhi!
4  print(mymodule.add(5, 7))        # 12
```

Using from-import in custom module

```
1  from mymodule import greet, add
2
3  print(greet("Python"))           # Hello, Python!
3  print(add(10, 20))               # 30
```

★ Keep code in separate files, import when needed.

5. Packages

- A package is a directory containing multiple modules and a special file __init__.py
- Directory structure example :

```
mypackage/
    __init__.py
    module1.py
    module2.py
```

Creating & Using Package

1. Create a folder (mypackage)
2. Add __init__.py inside it (can be empty)
3. Add .py files (modules) inside the folder
4. Import using dot notation

mypackage/module1.py

```
1  def hello():
2      return "Hello from module1!"
```

mypackage/module2.py

```
1  def square(x):
2      return x * x
```

main.py (using package)

```
1  from mypackage import module1, module2
2
3  print(module1.hello())     # Hello from module1!
4  print(module2.square(5))   # 25
```

★ Packages help in organizing related modules.

Note :
- __init__.py can contain initialization code for the package.
- It makes Python treat the directory as a package.

★ Quick Summary

- ☑ Use import to import entire module.
- ☑ Use from ... import to import specific items.
- ☑ Built-in modules are always available.
- ☑ Create your own modules and packages for clean structure.

Some Useful Built-in Modules at a Glance

math	random	datetime	os	sys	json
Trigonometry, log, constants, etc.	Random numbers, choices, shuffling	Date, time, timedelta, etc.	File operations, directories, paths	System info, arguments, etc.	JSON encode, decode, etc.

Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 12. EXCEPTION HANDLING

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

★ What is Exception Handling ?

- Exceptions are runtime errors that disrupt the normal flow of a program.
- Exception handling lets us handle those errors gracefully without crashing the program.
- It improves code reliability and user experience.

Handle exceptions gracefully, keep your program strong and user friendly ! ◡̈

1. try Block

- Code that may raise an exception is placed inside try block.
- If no exception occurs, rest of the code in try executes.

Example :

```
try:
    x = int(input("Enter a number: "))
    print("You entered:", x)
except ValueError:
    print("Invalid input! Please enter a number.")
```

★ Try the risky code, catch the errors ! ◡̈

2. except Block

- Used to handle the exception that occurs in try block.
- We can have multiple except blocks for different exceptions.

Example :

```
try:
    a = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero!")
except Exception as e:
    print("Something went wrong :", e)
```

★ Catch the right error, take the right action ! ◡̈

3. else Block

- Executes only if no exception occurs in try block.
- Useful for code that depends on successful execution.

Example :

```
try:
    num = int(input("Enter a number: "))
except ValueError:
    print("Not a valid number!")
else:
    print("Good! You entered", num)
```

★ Else runs only when everything is fine ! ◡̈

4. finally Block

- Executes always, no matter what (exception or not).
- Used for cleanup actions (closing files, releasing resources, etc.).

Example :

```
try:
    f = open("data.txt", "r")
    content = f.read()
except FileNotFoundError:
    print("File not found!")
finally:
    print("This will always execute.")
```

★ Finally is your cleanup buddy. Always runs ! ◡̈

★ Quick Summary

- ☑ try → Put risky code here.
- ☑ except → Handle the error.
- ☑ else → Run code if no error.
- ☑ finally → Always run (cleanup code).
- ☑ raise → Manually raise an error.
- ☑ Custom Exception → Create your own error.

Good code is not just about working when things go right, it's about handling when things go wrong ! ◡̈

Common Built-in Exceptions

Exception	Description
ZeroDivisionError	Division by zero.
ValueError	Invalid value for a function.
TypeError	Operation on incompatible types.
IndexError	Index out of range.
KeyError	Key not found in dictionary.
FileNotFoundError	File or path not found.
ImportError	Import statement fails.

You don't need to remember all exceptions. Handle the ones you may face ! ◡̈

5. raise Statement

- Used to manually raise an exception.
- We can raise built-in or custom exceptions.

Example :

```
age = -5
if age < 0:
    raise ValueError("Age cannot be negative!")
print("Age is", age)
```

★ Raise the error yourself when you detect something wrong ! ◡̈

Note :
- Program stops and jumps to the nearest except block.

6. Custom Exceptions

- We can create our own exception classes by extending Exception class.
- Useful for application-specific error handling.

Example : Create a custom exception

```
class InvalidAgeError(Exception):
    """Raised when age is not valid."""
    pass
```

Using Custom Exception

```
def check_age(age):
    if age < 0 or age > 120:
        raise InvalidAgeError("Age must be between 0 and 120")
    print("Age is valid:", age)

try:
    user_age = int(input("Enter your age: "))
    check_age(user_age)
except InvalidAgeError as e:
    print("Error:", e)
```

★ Custom exceptions make your code more meaningful and clear ! ◡̈

Note :
- All custom exceptions should inherit from Exception class.
- Use meaningful names for your exceptions.

Flow of Exception Handling

Scenario	Flow
No Exception	try block executes → else block (if present) → finally block
Exception Occurs	try block → except block(s) → finally block
raise Statement	raise executes → jumps to except → finally block
Exception in except	Goes to next except → if none matches → finally → then program stops

Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 13. FILE HANDLING

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

★ What is File Handling ?

- File handling allows us to store data permanently in files on disk.
- Python provides built-in functions to open, read, write, append and close files.
- Files can be text files, CSV files, JSON files etc.

Files help you save data between program runs !

1. Open & Close File

- open() opens a file and returns a file object.
- Always close the file using close() after use.

```
f = open('example.txt', 'r')   # open in read mode
print(f)                       # <_io.TextIOWrapper ...>
f.close()                      # close the file
```

★ Always close your files to free resources !

2. Read File

- Use read(), readline(), readlines().
- read() → entire content
- readline() → one line
- readlines() → list of lines

```
f = open('example.txt', 'r')
data = f.read()                # entire file
print(data)

f.seek(0)                      # move pointer to start
line1 = f.readline()           # first line
line2 = f.readline()           # second line

f.seek(0)
lines = f.readlines()          # list of all lines
print(lines)
f.close()
```

★ File pointer moves as you read. Use seek() to reset !

3. Write File

- write() writes a string to the file.
- If file does not exist, it will be created (in 'w' mode).

```
f = open('output.txt', 'w')
f.write("Hello, Python!\n")
f.write("This is file handling.")
f.close()
```

★ 'w' mode overwrites existing content !

4. Append to File

- Use 'a' mode to add content at the end of the file.
- File pointer is at the end by default.

```
f = open('output.txt', 'a')
f.write('\nAppended line here.')
f.close()
```

★ 'a' mode never deletes existing content !

Important Notes

- Use with statement → automatic close even if error occurs.
- File operations may raise exceptions (use try-except).
- Always specify encoding for text files when needed.
- Binary mode is useful for images, PDFs, etc.

Example: Using with (Recommended)

```
with open('example.txt', 'r') as f:
    data = f.read()
    print(data)
# File is automatically closed here
```

★ with is better and safer !

File Handling Flow

Open File (open()) → Perform Operation (read/write/append) → Process Data → Close File (close())

File Modes

Mode	Description	File Pointer	Creates File if Not Exists?
r	Read (default)	Start	No
w	Write (overwrites)	Start	Yes
a	Append (adds at end)	End	Yes
x	Create (fails if exists)	Start	Yes
b	Binary mode	-	-
t	Text mode (default)	-	-
+	Read and Write	Start	-

Choose the right mode for the right job !

5. Common File Methods

```
f.tell()          # current position of file pointer
f.seek(offset)    # move pointer to offset (default: from start)
f.truncate()      # truncate file at current position
f.close()         # close the file
```

6. Working with CSV Files

- CSV → Comma Separated Values (tabular data).
- Use csv module.
- Example:

```
import csv

# Writing to CSV
with open('data.csv', 'w', newline="") as f:
    writer = csv.writer(f)
    writer.writerow(['Name', 'Age', 'City'])
    writer.writerow(['Abhi', 20, 'Delhi'])
    writer.writerow(['Riya', 19, 'Mumbai'])

# Reading CSV
with open('data.csv', 'r') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Output

```
['Name', 'Age', 'City']
['Abhi', '20', 'Delhi']
['Riya', '19', 'Mumbai']
```

★ CSV is great for spreadsheets and tabular data !

7. Working with JSON Files

- JSON → JavaScript Object Notation (key-value data).
- Use json module.
- Example:

```
import json
data = {'name': 'Abhi', 'age': 20, 'city': 'Delhi'}

# Write JSON to file
with open('data.json', 'w') as f:
    json.dump(data, f, indent=4)

# Read JSON from file
with open('data.json', 'r') as f:
    data_read = json.load(f)
    print(data_read)
```

Output

```
{'name': 'Abhi',
 'age': 20,
 'city': 'Delhi'}
```

★ JSON is perfect for nested data and APIs !

Quick Summary

- ☑ Open file using open().
- ☑ Read using read(), readline(), readlines().
- ☑ Write using write().
- ☑ Append using 'a' mode.
- ☑ CSV → use csv module.
- ☑ JSON → use json module.
- ☑ Always close the file or use with.

Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 14. OOP (OBJECT ORIENTED PROGRAMMING)

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhiii07)

★ What is OOP ?

- OOP stands for Object Oriented Programming.
- It is a programming paradigm based on the concept of "Objects", which can contain data and code.
- It helps in writing clean, reusable and maintainable code.

Think of OOP as creating real-world entities in your programs !

Key OOP Concepts

Concept	Description
Class	A blueprint or template for creating objects.
Object	An instance of a class.
Constructor	A special method that initializes objects.
Attributes	Variables that belong to a class/objects.
Methods	Functions that belong to a class/objects.

OOP makes code modular, easy to manage and closer to real world !

1. Classes

- A class is a blueprint for creating objects.
- It defines the attributes and methods that objects will have.

Syntax :

```python
class Student:
    pass      # empty class

# Class with attributes and methods
class Student:
    school = "ABC School"  # class attribute (shared by all objects)

    def info(self):        # method
        print("This is a student class.")
```

★ Class is like a blueprint (design) and object is the real thing built from it !

2. Objects

- An object is an instance of a class.
- Each object can have different values for attributes.
- Syntax : object_name = ClassName()

Example :

```python
class Student:
    def __init__(self, name, age):
        self.name = name       # instance attribute
        self.age = age

    def display(self):
        print(f"Name: {self.name}, Age: {self.age}")

# Creating objects
s1 = Student("Abhi", 16)
s2 = Student("Riya", 15)

s1.display()      # Name: Abhi, Age: 16
s2.display()      # Name: Riya, Age: 15
```

★ Every object has its own copy of instance attributes !

3. Constructor (__init__)

- A constructor is a special method that is called automatically when an object is created.
- The constructor's name is always __init__().
- Used to initialize object attributes.

Example :

```python
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

b = Book("Python 101", "John Doe")
print(b.title)     #  Python 101
print(b.author)    #  John Doe
```

★ Constructor runs only once when the object is created !

4. Attributes

- Attributes are variables that belong to a class or an object.
- Two types :
  - Class Attributes (shared by all objects)
  - Instance Attributes (unique to each object)

Example :

```python
class Car:
    wheels = 4                 # class attribute

    def __init__(self, brand, color):
        self.brand = brand     # instance attribute
        self.color = color

c1 = Car("Toyota", "Red")
c2 = Car("Honda", "Blue")

print(c1.wheels)       # 4  (same for all)
print(c1.brand)        # Toyota
print(c2.brand)        # Honda
```

★ Instance attributes are object-specific, class attributes are shared !

5. Methods

- Methods are functions defined inside a class.
- They describe the behavior of objects.

Example :

```python
class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius * self.radius

c = Circle(5)
print("Area:", c.area())     # Area: 78.5
```

★ Methods perform actions and work with object data !

6. Accessing Attributes and Methods

- Use dot (.) operator to access attributes and methods.

Example :

```python
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print(f"Hello, I am {self.name}")

p = Person("Abhi")

print(p.name)      # Abhi
p.greet()          # Hello, I am Abhi
```

★ Use dot operator to interact with objects easily !

Mini Example (All Together)

```python
class Student:
    school = "ABC School"
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def info(self):
        return f"{self.name} is {self.age} years old."

s = Student("Abhi", 16)
print(s.school)    # ABC School
print(s.info())    # Abhi is 16 years old.
```

Practice makes perfect !

OOP in One Line

OOP lets us bind data (attributes) and functions (methods) together into a single unit called Class! and create instance (objects) of it.

Quick Summary

- ☑ Class → Blueprint
- ☑ Object → Real world entity
- ☑ Constructor (__init__) → Initializes objects
- ☑ Attributes → Data of object/class
- ☑ Methods → Functions of object/class
- ☑ Dot (.) → Access attributes/methods

Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 15. OOP CONCEPTS

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

★ What are OOP Concepts ?

- OOP (Object Oriented Programming) is based on 4 main concepts.
- These concepts help us to write better, reusable and maintainable code.

OOP Concepts make our code powerful and easy to manage ! ☺

Why are they Important ?

- ☑ Code Reusability
- ☑ Better Organization
- ☑ Flexibility and Extensibility
- ☑ Easy Maintenance

Build once, use many times, extend whenever needed ! ☺

1. Encapsulation

- Encapsulation is the process of wrapping data (variables) and code (methods) together as a single unit.
- It restricts direct access to some of the object's components.

Example :

```python
class BankAccount:
    def __init__(self, owner, balance):
        self.__owner = owner        # private attribute
        self.__balance = balance    # private attribute

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):          # public method
        return self.__balance

acc = BankAccount("Abhi", 1000)
acc.deposit(500)
print(acc.get_balance())            # 1500
print(acc.__balance)                # AttributeError
```

Encapsulation Hides internal details and shows only what is necessary. ☺

Key Points
- ★ Data Hiding
- ★ Access Control
- ★ Better Security

Note : Use single leading underscore (_) for "protected" (convention) and double leading underscore (__) for "private" (name mangling).

2. Inheritance

- Inheritance allows a new class (child) to inherit attributes and methods from an existing class (parent).
- It promotes code reusability.

Example :

```python
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print("Animal speaks")

class Dog(Animal):              # Dog inherits Animal
    def speak(self):            # Method Overriding
        print("Dog barks")

d = Dog("Buddy")
d.speak()                       # Dog barks
print(d.name)                   # Buddy
```

Animal (Parent) → Dog (Child)

Child class gets properties and behaviors of Parent class. ☺

Note : Use super() to access parent class members inside child class.
Example : super().__init__(name)

3. Polymorphism

- Polymorphism means "many forms".
- It allows methods to do different things based on the object.
- There are two types :
  1. Compile-time (Method Overloading) - Not supported in Python
  2. Run-time (Method Overriding) - Supported in Python

Example (Run-time Polymorphism) :

```python
class Shape:
    def area(self):
        pass
class Rectangle(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h
    def area(self):
        return self.w * self.h
class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14 * self.r * self.r

def print_area(shape):
    print("Area:", shape.area())

r = Rectangle(4, 5)
c = Circle(3)
print_area(r)       # Area: 20
print_area(c)       # Area: 28.26
```

How it Works ? Same function (print_area) works for different objects in different ways. ☺

Key Points
- ★ One interface
- ★ Multiple forms
- ★ Increases flexibility

4. Abstraction

- Abstraction hides the implementation details and shows only the essential features.
- We use Abstraction to focus on what an object does, not how it does.
- In Python, we use abc module and abstractmethod.

Example :

```python
from abc import ABC, abstractmethod

class Vehicle(ABC):                 # Abstract Base Class
    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):                 # Concrete Class
    def start(self):
        print("Car started")

class Bike(Vehicle):
    def start(self):
        print("Bike started")

c = Car()
c.start()                           # Car started
b = Bike()
b.start()                           # Bike started

# v = Vehicle()     # TypeError (cannot instantiate abstract class)
```

Abstraction Exposes only the important features and hides the unnecessary details. ☺

Key Points
- ★ Hides complexity
- ★ Improves maintainability
- ★ Helps in designing large systems

Note : Any class with at least one abstract method must override it, otherwise it cannot be instantiated.

Quick Comparison

Concept	Focus	Key Benefit
Encapsulation	Hiding data and methods	Security & Data Protection
Inheritance	Reusing existing code	Code Reusability
Polymorphism	Many forms of methods	Flexibility
Abstraction	Hiding implementation	Simplicity & Clarity

Remember

- ☑ Encapsulation → Protect data
- ☑ Inheritance → Reuse code
- ☑ Polymorphism → Do more with same interface
- ☑ Abstraction → Show only what is needed

☺

One Line Summary

- ★ Encapsulation → Wrap data and functions together.
- ★ Inheritance → Build new classes from existing ones.
- ★ Polymorphism → Same method, different behavior.
- ★ Abstraction → Focus on what, not how.

☺

Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 16. ADVANCED PYTHON

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

What is Advanced Python ?

- Advanced Python features help us write efficient, clean and powerful code.
- These topics are widely used in real-world projects, libraries and frameworks.

Master these and your Python skills level will jump to next level !

Why Learn These ?

- ☑ Write efficient & optimized code
- ☑ Reduce memory usage
- ☑ Handle complex problems easily
- ☑ Improve code readability & reusability

Advanced tools for building professional applications !

1. Iterators

- An iterator is an object that allows us to iterate (go through) over elements one at a time.
- It must implement two methods: `__iter__()` and `__next__()`.
- The `next()` method returns the next value. When no more items, it raises `StopIteration`.

Example :

```python
class CountDown:
    def __init__(self, start):
        self.start = start

    def __iter__(self):
        self.current = self.start
        return self

    def __next__(self):
        if self.current < 0:
            raise StopIteration
        val = self.current
        self.current -= 1
        return val

# Using iterator
for num in CountDown(3):
    print(num)
```

Output

```
3
2
1
0
```

Key Points

- ★ Iterable → object you can loop over (list, tuple, str, etc.)
- ★ Iterator → object that remembers state while iteration.

2. Generators

- Generators are a simple way to create iterators.
- They use `yield` keyword instead of `return`.
- Generators are memory efficient.

Example :

```python
def countdown(n):
    while n >= 0:
        yield n
        n -= 1

# Using generator
for num in countdown(3):
    print(num)
```

Output

```
3
2
1
0
```

Key Points

- `yield` pauses the function and returns a value.
- State is saved, resumes from where it left off.
- Great for large data or infinite streams.

Generator Expression

Similar to list comprehension but uses () instead of [].

Example : squares of numbers

```python
squares = (x*x for x in range(5))
for s in squares:
    print(s)
```

Output

```
0
1
4
9
16
```

3. Decorators

- A decorator is a function that takes another function and extends its behavior without permanently modifying it.
- Decorators are applied using `@decorator_name` syntax.

Example :

```python
def my_decorator(func):
    def wrapper():
        print("Before function call")
        func()
        print("After function call")
    return wrapper

@my_decorator
def say_hello():
    print("Hello, Python !")

say_hello()
```

Output

```
Before function call
Hello, Python !
After function call
```

Key Points

- ★ Enhance functionality.
- ★ Keep code DRY.
- ★ Common use: logging, authentication, timing, etc.

Decorator with Arguments

```python
def repeat(num):
    def decorator(func):
        def wrapper():
            for _ in range(num):
                func()
        return wrapper
    return decorator

@repeat(3)
def great():
    print("Hi!")
great()
```

Output

```
Hi!
Hi!
Hi!
```

Decorator Flow

@decorator applied on function
↓
Function passed to decorator
↓
Decorator returns new function
↓
New function is called

4. Regular Expressions (re module)

- Regex is a pattern used to match text.
- Python provides `re` module for regex operations.

Common Patterns

Pattern	Meaning	Example Match
.	Any character	a, 1, @
^	Start of string	^hello → hello world
$	End of string	world$ → hello world
*	Zero or more	ab*c → ac, abc, abbbc
+	One or more	ab+c → abc, abbbbbc
?	Zero or one	colou?r → color, colour
[abc]	Any one of set	[abc] → a or b or c
[^abc]	Not in set	[^abc] → any except a,b,c
\d	Digit [0-9]	\d → 0, 1, 2 ...
\w	Word char [a-zA-Z0-9_]	\w → a, Z, 9, _
\s	Whitespace	space, tab, newline

Examples

```python
import re

text = "My number is 12345 and email is test@example.com"

# Find all digits
print(re.findall(r'\d+', text))

# Find email
print(re.findall(r'[\w.-]+@[\w.-]+', text))

# Substitute
new_text = re.sub(r'\d+', '#####', text)
print(new_text)

# Split by whitespace
print(re.split(r'\s+', text))
```

Output

```
['12345']
['test@example.com']
My number is ##### and email is test@example.com
['My', 'number', 'is', '12345', 'and', 'email', 'is', 'test@example.com']
```

Quick Summary

- ☑ Iterators → Objects that iterate using `__iter__()` & `__next__()`.
- ☑ Generators → Use yield, memory efficient.
- ☑ Decorators → Modify/extend function behavior.
- ☑ Regex → Powerful text pattern matching with re module.

When to Use What ?

Topic	Use When
Iterator	When you need full control over iteration process.
Generator	When working with large data or infinite sequences.
Decorator	When you want to add functionality to functions.
Regex	When you need to search/validate/modify text patterns.

Pro Tips

- ☑ Prefer Generators over lists for large data.
- ☑ Use Decorators to keep your code clean and modular.
- ☑ Regex can be tricky - test patterns online (regex101.com).
- ☑ Practice more - these concepts get better with real projects.

Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 17. PYTHON LIBRARIES

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

What are Python Libraries ?

- Libraries are pre-written collections of functions and tools that help us solve complex problems easily.
- Python has a rich ecosystem of libraries for data science, machine learning, visualization and more.

Libraries save time and effort by providing powerful built-in functions !

Why Learn These Libraries ?

- ☑ Handle large data efficiently
- ☑ Perform complex analysis with few lines of code
- ☑ Create beautiful and informative visualizations
- ☑ Widely used in Data Science & AI industry

Master these libraries to become a Data Science Pro !

1. NumPy (Numerical Python)

- NumPy is used for numerical computing.
- It provides support for arrays, matrices and mathematical functions.

Example :

```python
import numpy as np

# Create an array
arr = np.array([1, 2, 3, 4, 5])

# 2D array (matrix)
mat = np.array([[1, 2, 3],
                [4, 5, 6]])

# Basic operations
print(arr)                  # [1 2 3 4 5]
print(arr.shape)            # (5,)
print(arr.dtype)            # int64
print(arr + 10)             # [11 12 13 14 15]
print(arr * 2)              # [ 2 4 6  8 10]
print(np.sum(arr))          # 15
print(np.mean(arr))         # 3.0
print(np.max(arr))          # 5
print(np.min(arr))          # 1
```

Key Features

- ★ Powerful N-dimensional array object (ndarray)
- ★ Mathematical functions
- ★ Broadcasting
- ★ Linear algebra
- ★ Random number generation

Common Functions

np.array()	Create array
np.zeros()	Array of zeros
np.ones()	Array of ones
np.arange()	Range of values
np.reshape()	Reshape array
np.sum()	Sum of elements
np.dot()	Dot product

2. Pandas (Data Analysis Library)

- Pandas is used for data manipulation and analysis.
- It provides DataFrame structure similar to Excel/SQL tables.

Example :

```python
import pandas as pd

# Create DataFrame
data = {
    'Name': ['Amit', 'Sneha', 'Ravi'],
    'Age': [25, 28, 22],
    'City': ['Delhi', 'Mumbai', 'Pune']
}
df = pd.DataFrame(data)
print(df)

# Basic operations
print(df.head())                # First 5 rows
print(df.info())                # Info about DataFrame
print(df.describe())            # Summary statistics
print(df['Age'].mean())         # Mean of Age column
print(df[df['Age'] > 23])       # Filter rows
print(df.sort_values('Age'))    # Sort by Age
```

Key Features

- ★ DataFrame & Series
- ★ Data cleaning & preprocessing
- ★ Handling missing data
- ★ Grouping & aggregation
- ★ Easy file I/O (CSV, Excel, SQL, JSON)

Common Functions

pd.read_csv()	Read CSV
pd.read_excel()	Read Excel
df.dropna()	Remove NA
df.fillna()	Fill NA
df.groupby()	Group by
df.merge()	Join tables
pd.concat()	Concatenate

3. Matplotlib (Data Visualization Library)

- Matplotlib is used for creating static, animated and interactive plots.
- It is the most fundamental plotting library in Python.

Example :

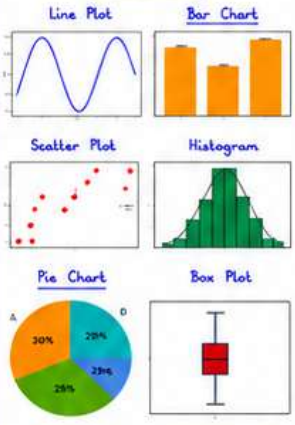
```python
import matplotlib.pyplot as plt
import numpy as np

# Data
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Line plot
plt.plot(x, y, label='sin(x)', color='b')
plt.title('Sine Wave')
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.legend()
plt.grid(True)
plt.show()

# Bar chart
categories = ['A', 'B', 'C']
values = [10, 20, 15]
plt.bar(categories, values, color='orange')
plt.title('Bar Chart')
plt.show()
```

Common Plots



Useful Functions plt.plot(), plt.scatter(), plt.bar(), plt.hist(), plt.pie(), plt.xlabel(), plt.ylabel(), plt.title(), plt.legend(), plt.grid(), plt.show()

4. Seaborn (Statistical Data Visualization)

- Seaborn is built on top of Matplotlib.
- It provides high-level interface for drawing attractive statistical graphics.

Example :

```python
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

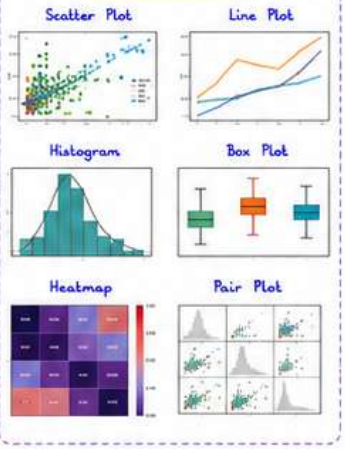
# Load built-in dataset
tips = sns.load_dataset('tips')

# Scatter plot with regression line
sns.scatterplot(data=tips, x='total_bill',
                y='tip', hue='sex')
plt.title('Scatter Plot with Seaborn')
plt.show()

# Histogram
sns.histplot(tips['total_bill'], kde=True, color='teal')
plt.title('Distribution of Total Bill')
plt.show()

# Box plot
sns.boxplot(data=tips, x='day', y='total_bill', palette='Set2')
plt.title('Total Bill by Day')
plt.show()
```

Seaborn Plot Gallery



Useful Functions sns.scatterplot(), sns.lineplot(), sns.histplot(), sns.boxplot(), sns.heatmap(), sns.pairplot(), sns.countplot(), sns.barplot()

Quick Summary

- ☑ NumPy → Work with arrays & mathematical operations
- ☑ Pandas → Work with data (clean, analyze, manipulate)
- ☑ Matplotlib → Create static plots & charts
- ☑ Seaborn → Create statistical & attractive visualizations

Tips

- ☑ Import libraries with aliases (np, pd, plt, sns)
- ☑ Use .head(), .info(), .describe() in Pandas
- ☑ Always label your plots (title, xlabel, ylabel)
- ☑ Seaborn makes statistical plots easier and prettier

Import Cheat Sheet

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 18. DATABASE CONNECTIVITY

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

★ What is Database Connectivity ?

- Database connectivity allows a Python program to store, retrieve and manage data in databases like SQLite, MySQL using SQL (Structured Query Language).

Learn SQL with Python!

Why Use Databases ?

- ☑ Store data permanently
- ☑ Retrieve and analyze data
- ☑ Handle large amounts of information
- ☑ Build dynamic applications

SQLite is lightweight and easy to use with Python!

1. SQLite - Basics

- SQLite is a lightweight, serverless, zero-configuration database engine.
- It stores data in a single file (`.db` file).
- Python's `sqlite3` module is used to work with SQLite.

```
import sqlite3
conn = sqlite3.connect("example.db")    # Open database
cur = conn.cursor()                     # Create cursor
```

- `conn.commit()` → Save changes.
- `conn.close()` → Close connection.

2. SQL - Basics

- SQL (Structured Query Language) is used to manage relational databases.
- Main SQL commands:

Command	Full Form	Purpose
SELECT	Select	Retrieve data
INSERT	Insert	Add data
UPDATE	Update	Modify data
DELETE	Delete	Remove data
CREATE	Create	Create tables

SQL is the language of databases.

3. CRUD Operations

Operation	SQL Command	Purpose
Create	CREATE TABLE	Create a table
Read	SELECT	Read data
Update	UPDATE	Modify data
Delete	DELETE	Remove data

4. Queries (Examples)

```
# Create Table
cur.execute("""
CREATE TABLE students (
  id INTEGER PRIMARY KEY,
  name TEXT,
  age INTEGER
)""")
```

```
# Insert
cur.execute("""
  INSERT INTO students
  (name, age)
  VALUES ('Rahul', 20)
""")
```

```
# Select
cur.execute("""
SELECT * FROM students
""")
```

```
# Update
cur.execute("""
UPDATE students
 WHERE name = 'Rahul'
""")
```

```
# Delete
cur.execute("""
UDLETE FROM students
 WHERE name = 'Rahul'
""")
```

```
# Fetch Results
rows = cur.fetchall()
  for row in rows:
      print(row)
```

SQL makes data handling powerful!

5. Common SQLite Methods

- `sqlite3.connect()` - Connect to database
- `cursor()` - Create cursor
- `execute()` - Run SQL query
- `fetchone()` - Fetch one record
- `fetchall()` - Fetch all records
- `commit()` - Save changes
- `close()` - Close database

Key Points:

★ Use parameterized queries to avoid SQL injection.

★ Always commit() after INSERT, UPDATE or DELETE.

6. CRUD in Action (with Python)

Create (INSERT)

```
cur.execute("INSERT INTO users
  (name, age) VALUES (?, ?)",
  ("Amit", 22))
conn.commit()
```

Read (SELECT)

```
cur.execute("SELECT * FROM users")
rows = cur.fetchall()
for row in rows:
    print(row)
```

Update (UPDATE)

```
cur.execute("UPDATE users
  SET age = ? WHERE name = ?",
  (23, "Amit"))
conn.commit()
```

Delete (DELETE)

```
cur.execute("DELETE FROM users
 WHERE name = ?", ("Amit",))
conn.commit()
```

7. SQL Query Types

Type	Example	Use
DDL	CREATE, ALTER, DROP	Structure DB
DML	INSERT, UPDATE, DELETE	Modify data
DQL	SELECT	Retrieve data
TCL	COMMIT, ROLLBACK	Transaction control

Transaction Example:

```
cur.execute("BEGIN")
cur.execute("UPDATE accounts SET balance = 1000 WHERE id = 1")
cur.execute("COMMIT")
```

8. SQLite Best Practices

- ☑ Always close the connection.
- ☑ Use parameterized queries (? placeholders).
- ☑ Commit changes after write operations.
- ☑ Handle exceptions (try-except).
- ☑ Use context manager for safety.

```
with sqlite3.connect("example.db") as conn:
    cur = conn.cursor()
    cur.execute("SELECT * FROM users")
```

9. Quick Reference

- Open DB: `conn = sqlite3.connect("file.db")`
- Create cursor: `cur = conn.cursor()`
- Fetch data: `rows = cur.fetchall()`
- Close DB: `conn.close()`

10. Interview Questions

1. What is SQLite?
2. Difference between SQLite and MySQL?
3. What are CRUD operations?
4. How do you prevent SQL injection?
5. Explain transactions in SQL.

11. Summary

- SQLite is a simple and powerful database for Python.
- SQL is used to perform Create, Read, Update and Delete operations on data.
- Use parameterized queries and commit() for safe and reliable database operations.

Remember: Python + SQL = Powerful Data Handling!

Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 19. PYTHON PROBLEM SOLVING

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

★ Why Problem Solving ?

- It builds logical thinking and pattern recognition.
- Helps in coding interviews and real-world apps.
- Common patterns appear repeatedly in problems.
- Master patterns → Solve problems faster and better.

Solve patterns, not just problems. Practice + Patterns = Power 💪

Key Takeaways

- ☑ Identify the pattern
- ☑ Choose the right approach
- ☑ Break the problem
- ☑ Solve → Optimize → Test

Good coders know patterns. Great coders master them !

1. Searching

Searching means finding the position (index) of a target element in a collection. The two most common searches are:

- Linear Search → Works on unsorted or sorted data.
- Binary Search → Works only on sorted data. (Faster)

A. Linear Search (O(n))

Check each element one by one.

```
def linear_search(arr, target):
    for i, val in enumerate(arr):
        if val == target:
            return i
    return -1    # Not found
```

Example:

arr = [4, 2, 7, 1, 9], target = 7

Output: 2

Complexity

Best Case : O(1)
Avg Case : O(n)
Worst Case : O(n)

B. Binary Search (O(log n))

Divide & conquer on sorted data.

```
def binary_search(arr, target):
    l, r = 0, len(arr) - 1
    while l <= r:
        mid = (l + r) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            l = mid + 1
        else:
            r = mid - 1
    return -1
```

Example:

arr = [1, 3, 5, 7, 9, 11, 13], target = 7

Output: 3

When to use ?

- ★ Linear → Small data or unsorted data
- ★ Binary → Large sorted data

Tips

- ☑ Always sort the data before Binary Search.
- ☑ Binary Search is much faster for large datasets.

2. Sorting

Sorting means arranging elements in a particular order.

Common sorting algorithms:

A. Bubble Sort (O(n²))

Repeatedly swap adjacent elements if they are in wrong order.

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):

        for j in range(0, n-i-1):

            if arr[j] > arr[j+1]:

                arr[j], arr[j+1] =
                arr[j+1], arr[j]
```

B. Selection Sort (O(n²))

Find min element and place it at correct position.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] =
        arr[min_idx], arr[i]

    return arr
```

C. Merge Sort (O(n log n))

Divide array into halves, sort and merge.

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr)//2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

    # merge() merges two
    # sorted arrays
```

When to use ?

- ★ Bubble / Selection → Small data
- ★ Merge Sort → Large data (Stable, Fast)

Tips

- ☑ Prefer built-in sorted() or .sort() in Python.
- ☑ Understand Time Complexity before choosing.

3. Two Pointers

Use two pointers (usually left & right) to solve problems in O(n) time.

Best for sorted arrays / pairs / palindromes / sum problems.

Pattern

1. Initialize two pointers
   left = start
   right = end
2. Move pointers based on condition
3. Shrink the window until solution found

Example: Pair with Target Sum (Sorted Array)

Find if any two numbers add up to target.

```
def pair_with_sum(arr, target):
    l, r = 0, len(arr) - 1
    while l < r:
        s = arr[l] + arr[r]
        if s == target:
            return (arr[l], arr[r])
        elif s < target:
            l += 1
        else:
            r -= 1
    return None
```

Example Walkthrough

arr = [2, 4, 6, 8, 10, 14],
target = 16

l ↓					r ↓
2	4	6	8	10	14

2+14=16 ✓

Output: (2, 14)

Common Uses

- ★ Pair Sum / Triplet Sum
- ★ Palindrome Check
- ★ Remove Duplicates
- ★ 3Sum, 4Sum Problems

Complexity

Time : O(n)
Space : O(1)

Tips

- ☑ Works best on sorted arrays.
- ☑ Move the pointer that gives a better chance.

4. Sliding Window

Used for arrays / strings to keep a "window" (subarray/substring) and adjust it to get the result in O(n) time.

Pattern

1. Expand right pointer to include elements
2. Shrink left pointer when condition breaks
3. Track result while moving window

Example: Maximum Sum Subarray of Size K

Find maximum sum of subarray of size k.

```
def max_sum_k(arr, k):
    n = len(arr)
    window_sum = sum(arr[:k])
    max_sum = window_sum
    for i in range(k, n):
        window_sum += arr[i] - arr[i-k]
        max_sum = max(max_sum, window_sum)
    return max_sum
```

Example Walkthrough

arr = [2, 1, 5, 1, 3, 2], k = 3

2 1 5 1 3 2
Window [0..2], sum = 8

1 5 1 1 3 2
Window [1..3], sum = 7

5 5 1 3 2
Window [2..4], sum = 9

1 1 3 2
Window [3..5], sum = 6

Max Sum = 9

Common Uses

- ★ Max/Min Sum Subarray of Size K
- ★ Longest Substring Without Repeating
- ★ Count of Anagrams in String

Complexity

Time : O(n)
Space : O(1)

Tips

- ☑ Expand window
- ☑ Shrink when invalid
- ☑ Keep updating answer

Quick Comparison

Technique	Best For	Time Complexity	Space Complexity
Linear Search	Any data	O(n)	O(1)
Binary Search	Sorted data	O(log n)	O(1)
Bubble Sort	Small data	O(n^2)	O(1)
Selection Sort	Small data	O(n^2)	O(1)
Merge Sort	Large data	O(n log n)	O(n)
Two Pointers	Sorted / Pairs	O(n)	O(1)
Sliding Window	Subarray / Substring	O(n)	O(1)

Problem Solving Strategy

1. Understand the problem
2. Identify the pattern
3. Choose the right technique
4. Write dry run / test cases
5. Code and test
6. Optimize if needed

Practice Problems (Try These!)

- ☑ Search: Find target in sorted array (Binary Search)
- ☑ Sort: Sort an array of 0s, 1s and 2s
- ☑ Two Pointers: 2 Sum in sorted array
- ☑ Two Pointers: Valid Palindrome
- ☑ Sliding Window: Longest substring without repeating characters
- ☑ Sliding Window: Max sum subarray of size K

Think Smart. Code Better. Optimize Always !

PYTHON NOTES - 20. PYTHON PRACTICE

By Abhishek Rathor
(ig : SYNTAX ERROR || code.abhii07)

★ Why Practice ?

- Practice makes concepts strong.
- Helps in thinking logically and writing better code.
- Important for interviews and real-world problems.
- The more you practice, the more confident you become !

Code. Test. Debug. Repeat. That's how you level up !

Key Takeaways

- ☑ Practice daily - small steps, big progress
- ☑ Solve a variety of problems
- ☑ Think before coding
- ☑ Learn from mistakes
- ☑ Consistency is the key to success

Consistency Today, Expert Tomorrow !

1. Basic Programs

Simple programs to build strong basics.

Program	Code	Output
1. Print Hello, World!	`print("Hello, World!")`	Hello, World!

2. Add Two Numbers	`a = 10`
	`b = 20`
	`print(a + b)`
		30

3. Check Even or Odd	`n = int(input())`
	`if n % 2 == 0:`
	`  print("Even")`
	`else:`
	`  print("Odd")`
		Input: 8
		Output: Even
		Input: 7
		Output: Odd

4. Factorial of a Number	`n = int(input())`
	`fact = 1`
	`for i in range(1, n+1):`
	`  fact *= i`
	`print(fact)`
		Input: 5
		Output: 120

5. Fibonacci Series	`n = int(input())`
	`a, b = 0, 1`
	`for _ in range(n):`
	`  print(a, end=" ")`
	`  a, b = b, a + b`
		Input: 6
		Output:
		0 1 1 2 3 5

2. Pattern Programs

Improve logic and loop understanding.

1. Right Triangle

```
n = 5
for i in range(1, n+1):
    print("*" * i)
```

Output:

```
*
**
***
****
*****
```

2. Inverted Triangle

```
n = 5
for i in range(n, 0, -1):
    print("*" * i)
```

Output:

```
*****
****
***
**
*
```

3. Pyramid

```
n = 5
for i in range(1, n+1):
    print(" "*(n-i)+"*"*(2*i-1))
```

Output:

```
    *
   ***
  *****
 *******
*********
```

4. Number Pyramid

```
n = 5
for i in range(1, n+1):
    print(" "*(n-i), end="")
    for j in range(1, i+1):
        print(j, end=" ")
    print()
```

Output:

```
    1
   1 2
  1 2 3
 1 2 3 4
1 2 3 4 5
```

5. Floyd's Triangle

```
n = 5
num = 1
for i in range(1, n+1):
    for j in range(1, i+1):
        print(num, end=" ")
        num += 1
    print()
```

Output:

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

3. Output Questions

What will be the output of the following code?

Q1.

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

Answer:
[1, 2, 3, 4]

Q2.

```
x = 10
def fun(x):
    x = 20
fun(x)
print(x)
```

Answer:
10

Q3.

```
for i in range(3):
    print(i, end=" ")
else:
    print(i)
```

Answer:
0 1 2 2

Q4.

```
s = "hello"
print(s[::-1])
```

Answer:
olleh

Q5.

```
d = {'a': 1, 'b': 2}
print(d.get('c', 'NA'))
```

Answer:
NA

Q6.

```
print(2 ** 3 ** 2)
```

Answer:
512
(2 ** (3 ** 2) = 2 ** 9)

Q7.

```
l = [i for i in range(5)
       if i % 2 == 0]
print(l)
```

Answer:
[0, 2, 4]

Q8.

```
a = (1, 2, 3)
b = list(a)
b[1] = 20
print(tuple(b))
```

Answer:
(1, 20, 3)

Q9.

```
s = set([1, 2, 2, 3])
s.add(4)
s.remove(2)
print(s)
```

Answer:
{1, 3, 4}

4. Interview Questions (Theory)

No.	Question	Answer (Short)
1	What is Python?	High-level, interpreted, general-purpose programming language.
2	What is the difference between list and tuple?	List is mutable (can change), tuple is immutable (cannot change).
3	What is the difference between '==' and 'is'?	'==' compares values, 'is' compares memory location.
4	What is a dictionary in Python?	A collection of key-value pairs (unordered, mutable).
5	What is the use of 'self' in Python?	Refers to the instance of a class.
6	What is a lambda function?	Anonymous function written in a single line.
7	What is the difference between 'append()' and 'extend()'?	append() adds one item, extend() adds items from iterable.
8	What is recursion?	A function calling itself to solve smaller subproblems.
9	What is a module?	A Python file containing functions, classes, variables.
10	What is PEP 8?	Python Enhancement Proposal 8 - Style guide for Python code.

5. Interview Questions (Coding)

No.	Question	Sample Approach / Logic
1	Write a program to find factorial of a number.	Use loop or recursion.
2	Write a program to check if a number is prime.	Check divisibility from 2 to sqrt(n).
3	Write a program to reverse a string.	Use slicing (s[::-1]) or loop.
4	Write a program to find largest element in a list.	Initialize max and traverse list.
5	Write a program to remove duplicates from a list.	Convert list to set, then back to list.
6	Write a program to count vowels in a string.	Check each character in 'aeiouAEIOU'.
7	Write a program to find second largest in a list.	Remove max, then find max again.

6. Mini Practice Set (Try These!)

1. Print multiplication table of n.
2. Take n numbers and print their sum.
3. Check if a string is palindrome.
4. Print common elements of two lists.
5. Merge two sorted lists.
6. Find frequency of each element in a list.
7. Convert decimal to binary.
8. Count words in a sentence.
9. Sort a list without using sort().
10. Find missing number in range 1 to n.

7. Challenge Programs

- ★ Implement binary search.
- ★ Solve the Tower of Hanoi problem.
- ★ Check for anagram strings.
- ★ Find longest common prefix.
- ★ Implement LRU Cache (basic version).
- ★ Rotate matrix by 90 degrees.
- ★ Find all substrings of a string.
- ★ Solve N-Queens problem.

Time Complexity Guide

O(1)	Constant
O(log n)	Logarithmic
O(n)	Linear
O(n log n)	Linearithmic
O(n^2)	Quadratic
O(2^n)	Exponential

Tips to Improve

- ☑ Solve daily (at least 1-2 problems).
- ☑ Understand before you memorize.
- ☑ Write code by yourself.
- ☑ Test edge cases.
- ☑ Review and analyze your solutions.

Best Platforms to Practice

- ★ LeetCode (leetcode.com)
- ★ HackerRank (hackerrank.com)
- ★ Codeforces (codeforces.com)
- ★ GeeksforGeeks (geeksforgeeks.org)
- ★ InterviewBit, Coding Ninjas, Excersize

Practice today, Perfect tomorrow, Perform forever !

Think Smart. Code Better. Optimize Always !